



MusiKeyrtual: A Framework to Play a Musical Keyboard in Augmented Reality

Valerio Venanzi, Andrea Princic, Marco Raoul Marini, Gian Luca Foresti & Luigi Cinque

To cite this article: Valerio Venanzi, Andrea Princic, Marco Raoul Marini, Gian Luca Foresti & Luigi Cinque (24 Sep 2025): MusiKeyrtual: A Framework to Play a Musical Keyboard in Augmented Reality, International Journal of Human-Computer Interaction, DOI: [10.1080/10447318.2025.2561764](https://doi.org/10.1080/10447318.2025.2561764)

To link to this article: <https://doi.org/10.1080/10447318.2025.2561764>



Published online: 24 Sep 2025.



Submit your article to this journal [↗](#)



View related articles [↗](#)



View Crossmark data [↗](#)



MusiKeyrtual: A Framework to Play a Musical Keyboard in Augmented Reality

Valerio Venanzi^a , Andrea Princic^a , Marco Raoul Marini^a , Gian Luca Foresti^b  and Luigi Cinque^a 

^aDepartment of Computer Science, Sapienza University of Rome, Rome, Italy; ^bDepartment of Mathematics, Informatics and Physics, University of Udine, Udine, Italy

ABSTRACT

The advances in Machine and Deep Learning (ML and DL, respectively) contribute to the evolution of modern Augmented Reality (AR) systems, adapting software to complex and interesting applications. Moreover, lower-end systems, such as smartphones, are now capable of running AR applications, albeit often requiring smaller and lighter DL and ML models to accommodate hardware limitations. In this context, we propose MusiKeyrtual, a lightweight application that allows users to play a musical keyboard drawn on paper; an improvement over the previous version, Keyrtual. The application requires only a smartphone to run. The pipeline proposed addresses the hardware limitations of smartphones, both in terms of limited computational capabilities and in terms of using a single RGB camera, which cannot detect depth. Quantitative and qualitative results highlight the effectiveness of the proposed pipeline in exploiting the capabilities of modern smartphones.

KEYWORDS

Computer vision; human-computer interaction; hand tracking; hand gesture; virtual musical instrument

1. Introduction

Deep Learning (DL) is a branch of Machine Learning (ML) that has gained a lot of attention from both researchers and the public, thanks to the promising results obtained in recent years. Many different DL models have been proposed, which have proven to be both useful and versatile, with practical applications in various fields; albeit needing extensive, and often specific datasets for training (Deng et al., 2009; Gil-Martín et al., 2024; Zottin et al., 2024). Deep Learning has been employed for image processing (Archana & Jeevaraj, 2024), medical image analysis (Kumar et al., 2023; Sarao et al., 2023) and audio recognition (Prashanth et al., 2024), among others. For a comprehensive survey, see Younesi et al. (2024).

One of the most prominent uses for DL and ML models is human-body tracking in general, often in the more specific cases of hand tracking and hand gesture recognition (Lugaresi et al., 2019). For these tasks, even simpler ML models achieved remarkable results, marking exceptional steps forward in the area of Human-Computer Interaction (HCI).

In Virtual Reality (VR), users dive into a virtual world with the help of special visors. Among the applications are medical training (Marini et al., 2024), rehabilitation (Avola et al., 2019), and even in retail, to enhance the shopping experience (Wohlgenannt et al., 2020). Akin to VR is Augmented Reality (AR), with many applications and drawbacks (Arena et al., 2022). There exist applications that require multiple and heterogeneous devices in order to work; the integration of said devices is complex and often limited. The use of heavy DL models requires hardware capable of running them, which is expensive and often energy-consuming. Another great issue is privacy; applications, especially those that require an internet connection, should guarantee security mechanisms to ensure a safe and secure session for every user while using the application.

The cost of using DL architectures is the computational power needed for inference, requiring dedicated Graphics Processing Units (GPUs) to run large models. Training also requires additional training

time upfront, in addition to high-quality and adequately large datasets. Progress has been made to make both training (Fontana, Lanzino, Diko, et al., 2024) and ML models (Lanzino et al., 2024; Fontana, Lanzino, Marini, et al., 2024) easier to run.

In recent years, smartphones have also seen a surge in hardware capabilities, making it possible to run appropriate DL models directly on smartphones (Deng, 2019).

Musikeyrtual is a pipeline that combines classical computer vision algorithms with MediaPipe (Lugaresi et al., 2019) to track hand movements and play a piano keyboard drawn on paper. The pipeline is thought of as a solution “from the shelf” for a smartphone, without the requirement of any external cameras, nor does it delegate computation to external devices. The pipeline uses only the main RGB camera of the smartphones. It is worth noting that there exist smartphones with hardware capable of detecting depth (e.g., Samsung S20 Ultra), but they are not common; also, the precision of these sensors is subpar, and API integration is often limited, making it challenging to develop an application that effectively exploits these sensors.

One of the final goals of MusiKeyrtual is to create a more comprehensive experience, compared to using a pre-made piano, in which the user first creates their piano by drawing it, and then plays it. We believe that this would be particularly suited to elementary school kids, as a way of approaching various subjects: music, technical drawing, and possibly computer science. In particular, it is worth believing that an interaction on a wide surface, i.e., an A4 size format paper, should be more comfortable than a smartphone screen in terms of dimensions. In fact, a real-size piano keyboard (1 octave) perfectly fits the A4 paper, while on the smartphone, there is not enough space, even on foldable devices.

The document is structured as follows. In [Section 2](#), an overview of related works and state of the art; in [Section 3](#), the explanation of the pipeline; in [Section 4](#), an overview of the hardware used and the results achieved; [Section 6](#) ends with some final analysis and future prospects.

2. Related work

Recognizing humans in still images and videos has been the interest of many researchers for over 20 years (Foxlin & Harrington, 2000; Zhou & Hoang, 2005). Much of the effort goes towards the problems of hand recognition and gesture detection, with many applications (Guo et al., 2021; Mohamed et al., 2021; Qi et al., 2024).

Nowadays, in computer vision, gesture recognition is performed with Deep Learning models. Simpler ML models have been used in the past, see for instance De Smedt et al. (2016), where the authors classified gestures using a Support Vector Machine. In another work (Jasim et al., 2014), the authors used the k-Nearest Neighbours to classify hand gestures. Classical ML models often lack robustness (Qi et al., 2024), and thus improvements have been made using DL models. A lot of promising results come from attention-based networks. In Zhang et al. (2018), a convolutional Long short-term memory (LSTM) architecture, based on attention, has been used to perform gesture recognition, achieving good results. Convolutional Neural Networks (CNN) have also been deeply studied; in Jiang et al. (2019), a hand skeleton-based algorithm has been used on a CNN to recognize the American Sign Language (ASL) (Caselli et al., 2017).

Many architectures are often meant to work with RGB images, though in some cases one might also need depth (i.e., RGB-D images). Inferring depth from an RGB image is a difficult task (Masoumian et al., 2022); thus, dedicated hardware is often used to directly capture RGB-D images. Having in-depth information is crucial in the virtualization of musical instruments. In Liang et al. (2016), the authors combined RGB-D data with finger pose and tap detection to create a virtual piano. Hands might also be simulated in a 3D environment, using an Oculus Rift and a Leap Motion (Fillwalk, 2015). The shortcoming of using RGB-D cameras is the need for dedicated hardware, which not only makes the setup more costly, but also, in general, less accessible and often less portable, since many of these cameras often need to be connected to a computer to work.

The aim of this work is to create a pipeline that allows a user to play a piano drawn on paper, using just a smartphone, making the application portable and cheap. This work is an improvement of Avola et al. (2023), where the task is the same, but has been tackled using a combination of both an RGB and an RGB-D camera, from a laptop.

3. Proposed method

The goal of the pipeline is to allow users to play a keyboard drawn on paper. The software requires only a smartphone to run. Having such a target device means that the algorithms cannot be computationally expensive, and the limitations of using a single RGB camera must be addressed. In Figure 1, the architecture of the system is shown. In the Preliminary Phase, keyboard detection is performed to find the piano keyboard, and then during the Real-Time Phase, the user can play the keyboard.

Each one of the functions used was invoked from the OpenCV library.¹

3.1. Preliminary phase

The Preliminary Phase is the first in the pipeline, and its goal is to find and recognize the piano drawn on paper, using a module called Keyboard Detector. The Keyboard Detector used in this work is an improved version of the one in Avola et al. (2023).

The keyboard-detecting procedure starts by placing the smartphone above the keyboard to frame it at around a 90° angle (as shown in Figure 2(a)). A rectangle, namely overlay (Figure 3(a)), is shown on the screen, and the sheet of paper must be framed inside the area described by the overlay; it is going to be used to mask (Figure 3(b)) the area of the keyboard (Figure 3(c)). Masking the keyboard means

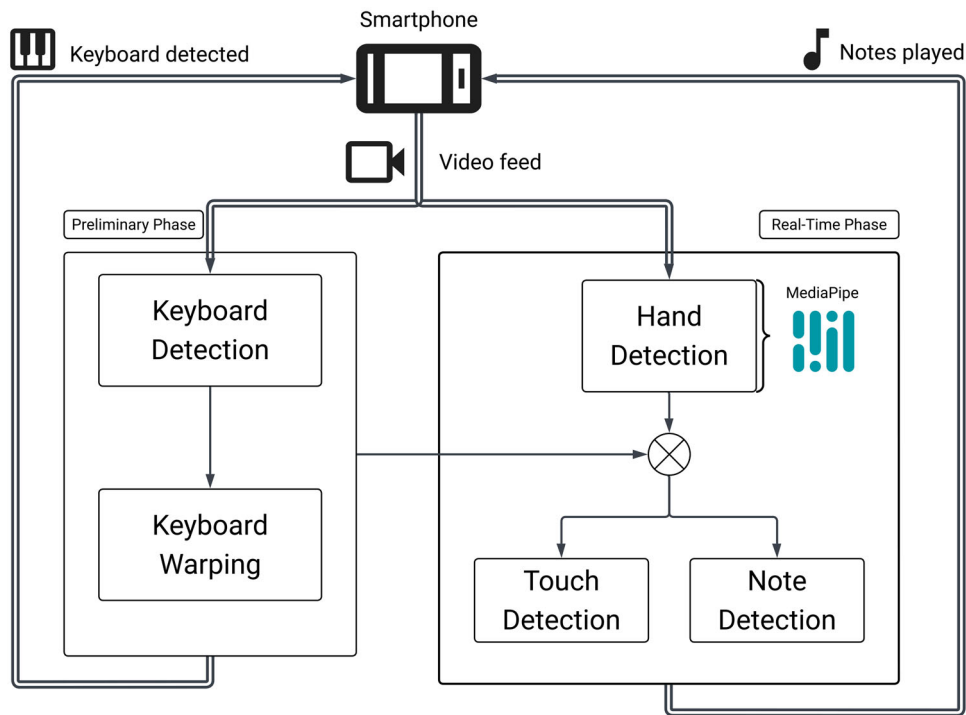


Figure 1. Architecture of the pipeline.



(a) Keyboard from above

(b) Keyboard from below

Figure 2. Keyboard as seen from the two perspectives. (a) is used to detect the keyboard, and (b) is used to play the keyboard.

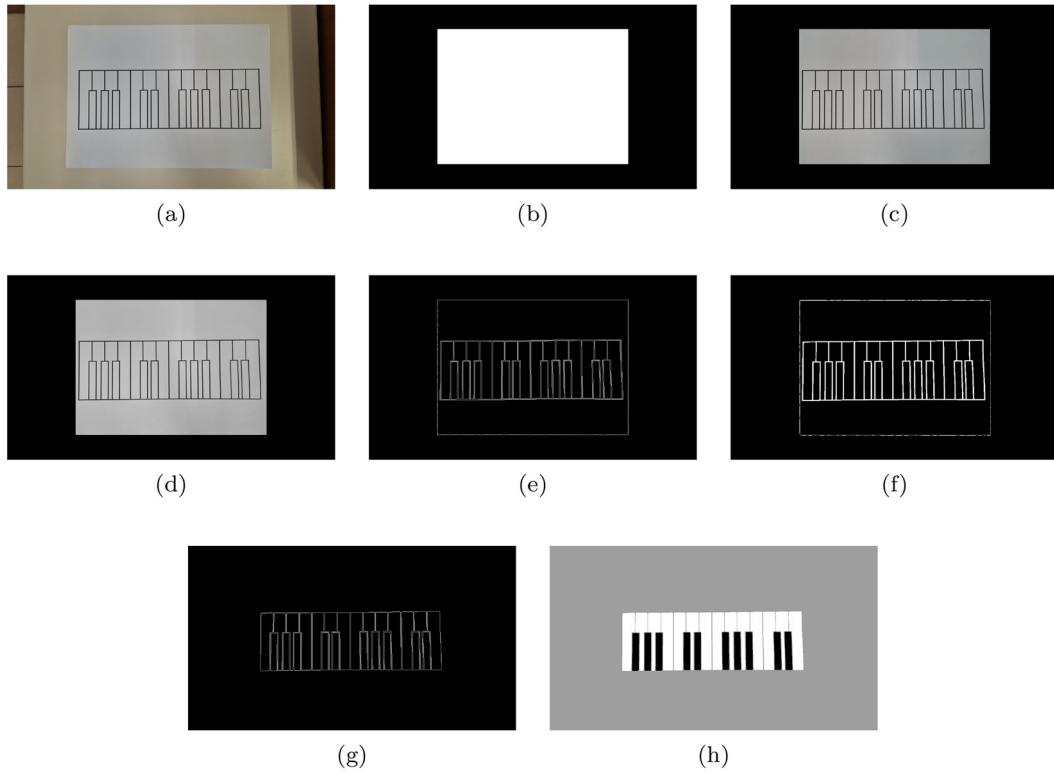


Figure 3. Preliminary phase pipeline. (a) Picture of the keyboard, (b) mask made with the overlay, (c) masked keyboard, (d) grayscale, (e) canny edge detection, (f) closing operator, (g) find contours, (h) keys detected.

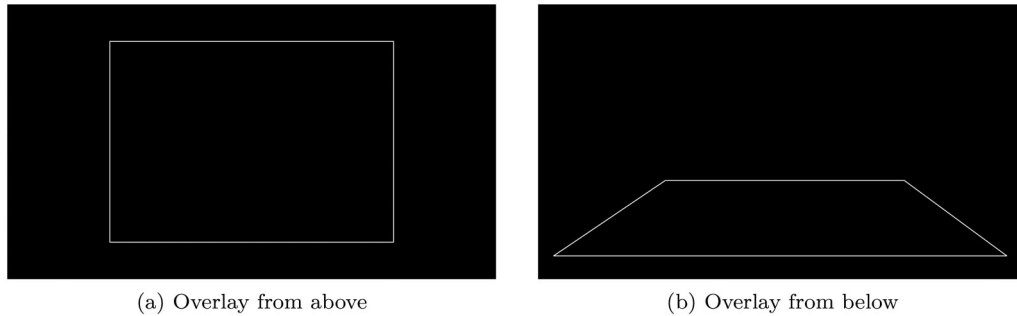


Figure 4. Overlays used to frame and mask the keyboard drawn on paper.

that nothing else is present on the image, making keyboard detection easier and the algorithm more robust. After masking, the keyboard image is resized to 1280×720 pixels.

The next goal is to find all the lines that compose the keyboard. The masked image is converted to grayscale (Figure 3(d)), and the Canny edge detector (Canny (1986)) is applied (Figure 3(e)); the output is then provided to a Closing operator to clean up the spikes on the edges found with Canny (Figure 3(f)). Following, the procedure to find contours (Suzuki & Be, 1985) is applied (Figure 3(g)). Contours are now grouped according to their height and then sorted by height. Given the point where the difference between a contour i and a contour $i + 1$ is maximum is the one that divides black-and-white keys. A sample of the final result of the detected keyboard is shown in Figure 3(h) (gray background applied to highlight both black and white keys).

Following, there is a change of point-of-view, and the smartphone is placed in front of the keyboard at around $10^\circ \sim 20^\circ$ angle from the lying surface of the sheet of paper (Figure 2(b)). This change of perspective is crucial during the Real-Time Phase, deeply explained in Section 3.2.1.

During the point-of-view modification, some operations are performed to keep track of the keyboard and map it from the original position to the novel one (shown in Figure 2(b)). Another overlay-based

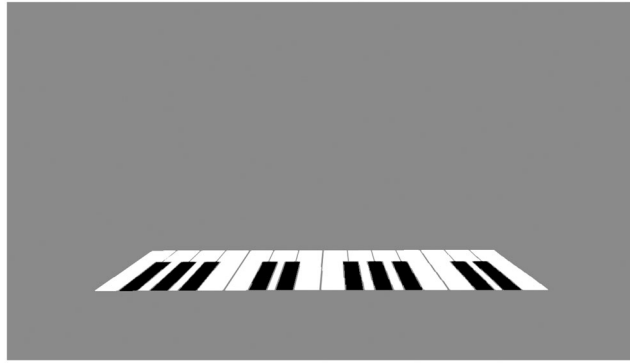


Figure 5. Keyboard warped in the new perspective.

approach, as seen in [Figure 4\(b\)](#), is used to mask the keyboard. Then, to match the keyboard from above with the keyboard from below, the Homography (Fischler & Bolles (1981) function is invoked, which finds the perspective transformation from the “above” perspective to the “below” one. A warp perspective operation is then applied to the previously detected keyboard, which gets transformed according to the form shown in [Figure 5](#) (gray background applied to highlight both black and white keys).

At the end of this phase, the software can manage the keys’ positions from the current viewpoint; the location of each of them is known. The Real-Time Phase is the following step.

3.2. Real-time phase

The Real-Time Phase allows the user to play the keyboard. During this phase, the software performs two sub-tasks: the **touch detection**, to detect if a finger is touching the keyboard; when it happens, the **note detection** algorithm is invoked to recognize which note is being played.

As previously mentioned, hand tracking is performed by exploiting the MediaPipe library, which infers a skeleton of the hand with 21 landmarks in real-time. These are points of interest of the hand, e.g., joints of the fingers. Only five landmarks, corresponding to the five fingertips of a hand, are effectively used to perform both the touch detection and the note detection.

3.2.1. Touch detection

This step of the pipeline is one of the most critical. A wrong finger touch action detection determines the incorrect execution of the real-time keyboard playing task. The smartphone is placed in front of the keyboard (as shown in [Figure 2\(b\)](#)); this viewpoint simplifies the Touch Detection task, making it easier to perform, considering the computational limitations of a smartphone. This positioning grants the best view of the hands to detect the vertical movement of the fingers; thus, the software relies only on the y position on the screen of each fingertip to detect if a finger is touching.

Given a finger, it might be performing one of four distinct actions, as depicted by the automata in [Figure 6](#). One automata is created for each finger.

State `idle` represents the finger staying still, above the piano. When it starts going down, the automata goes into state `going down`, and remains in this state until the finger has slowed down enough; in this case, it goes into state `touching`, which represents the window of frames in which a finger is touching the piano. A finger might be touching the paper but outside the piano; this means touching but not playing any note; the Note Detection algorithm deals with this case (see [Algorithm 2](#)) successively in the pipeline. When the finger stops playing a note, it starts moving up, away from the keyboard, and the automata goes into state `going up` (which will eventually go back to `idle` when the finger stops moving).

Consider v as the velocity value of the finger, which is computed by taking the difference between the finger’s position during frame $\mathcal{T}$ and its position during frame $\mathcal{T} - 1$. The value t is a small threshold used to switch state when a finger moves by at least t pixels from one frame to the next one; it helps mitigate MediaPipe’s instability by avoiding switching state when not needed (i.e., avoid false positives). The hand skeleton detected by MediaPipe is subject to continuous small movements, similar to a slight tremor of the hand, which would induce the automata to continuously change state. We

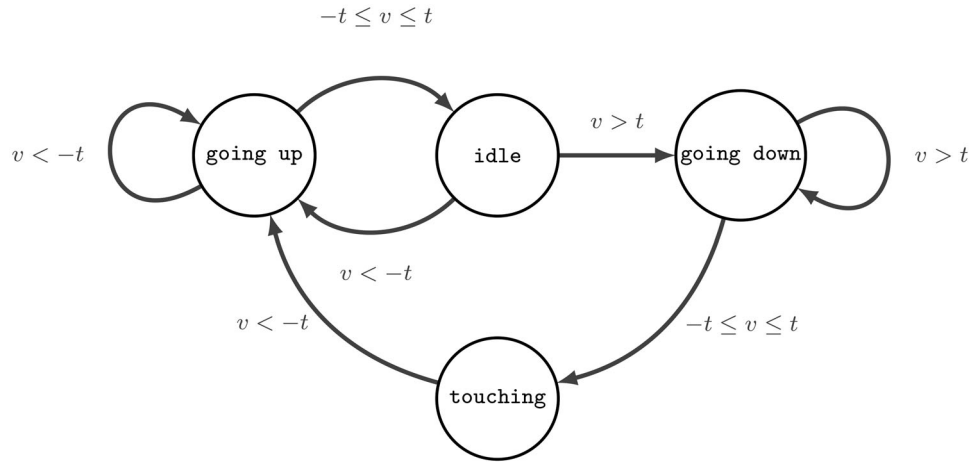


Figure 6. Automata detecting a finger's movement.

empirically retrieved that a value of $t = 8$ is suitable for most of the use cases (e.g., different smartphones and illumination conditions); a value smaller than 8 makes the automata switch state when not needed (too many false positives), while a value greater than 8 makes the automata too resistant to switching state, even when it should (too many false negatives). In other words, the combination of v and t makes the automata switch state if a finger produces a movement of at least 8 pixels.

The procedure for Touch Detection is shown in [Algorithm 1](#). The algorithm is invoked sequentially for each new frame.

Algorithm 1 Touch Detection

```

let  $H$  be the set of hands detected  $\triangleright |H| \in \{0, 1, 2\}$ 
if no hand has been detected then
    return
end if
 $notes \leftarrow \emptyset$ 
for each hand  $h \in H$  do
    for each finger  $f$  of hand  $h$  do
         $x \leftarrow$  horizontal position of  $f$ 
         $y \leftarrow$  vertical position of  $f$ 
        let  $A_f$  be the automata of finger  $f$ 
        update  $A_f$  with position  $y$ 
        if  $A_f$  is in state touching then
             $notes \leftarrow notes \cup \{(x, y)\}$ 
        end if
    end for
end for
call Note Detection algorithm with  $notes$ 

```

3.2.2. Note detection

After the Touch Detection phase, the algorithm detects which fingers are touching the piano, meaning they are playing some notes. Thus, Note Detection is invoked, the algorithm that detects which notes are being played to reproduce the sound on the device.

Note Detection procedure is shown in [Algorithm 2](#). The variables *detected*, *playing*, *stop* and *start* are mathematical sets. Each note can enter in one of the following three states:

- Was already playing and has been found: must not do anything, it keeps playing;

- Was already playing and has not been found: it's a note that the user stopped playing, must stop reproducing it;
- Was not playing and had been found: it's a new note that the user started playing, must start reproducing it.

These behaviors are ensured by the way the sets *stop* and *start* are defined in the algorithm.

Note also that the set *detected* contains only notes corresponding to some coordinates found by Algorithm 1. It means that when a finger is touching the piece of paper outside the piano (i.e., it is not inside any note's boundary), the touch is still correctly recognized, but the coordinates do not belong to any note; thus, the trigger does not start.

Algorithm 2 Note Detection

Require: *notes*, set of coordinates touched by fingers found by the Touch Detection procedure

Ensure: correct notes are reproduced at each frame

$detected \leftarrow \{n \mid \exists (x, y) \in notes \text{ s.t. } n \text{ corresponds to the note at coordinates } (x, y)\}$

$playing \leftarrow \{n \mid \text{note } n \text{ was already playing during the previous frame}\}$

$stop \leftarrow detected \setminus playing$

$start \leftarrow playing \setminus detected$

for each $n \in stop$ **do**

 stop playing note n

end for

for each $n \in start$ **do**

 start playing note n

end for

4. Experiments

This section is dedicated to the experiments performed to evaluate the performance of the pipeline. Various tests have been performed in order to analyze each component of the whole architecture and get a numerical evaluation. Additional tests have been carried out with various users to get results on the perceived usability of the system.

4.1. Experimental setup

Experiments have been made with various piano keyboards (as explained in Section 4.2), on A4 size paper. The smartphone used is a Samsung Galaxy S24.² The software has been developed with Unity.³

4.2. Keyboard detection

This test aims to evaluate how many pixels, corresponding to the keyboard, are correctly recognized. Experiments were carried out on three different types of keyboard: a (free) hand-drawn keyboard, one hand-drawn using a ruler, and a precise keyboard drawn with software and later printed out. The purpose of these tests is to both study the robustness of the Keyboard Detection procedure, which has not been previously studied in the previous work, and to highlight the capability of the procedure in detecting a keyboard even when the drawing has many imperfections, especially if hand-drawn.

The three types of keyboards used are shown in Figure 7.

The tests were carried out in a room with artificial light coming from the ceiling and natural light coming from an open window. For each of these keyboards, the test was carried out by changing the rotation and position of the keyboard with respect to the natural light 4 times, performing the keyboard detection 3 times for each position, for a total of 12 detections per keyboard type and 36 detections in total.

The results of the keyboard detector have been compared with the ground truth, obtained by manually selecting pixels that would belong to the keyboard or not.

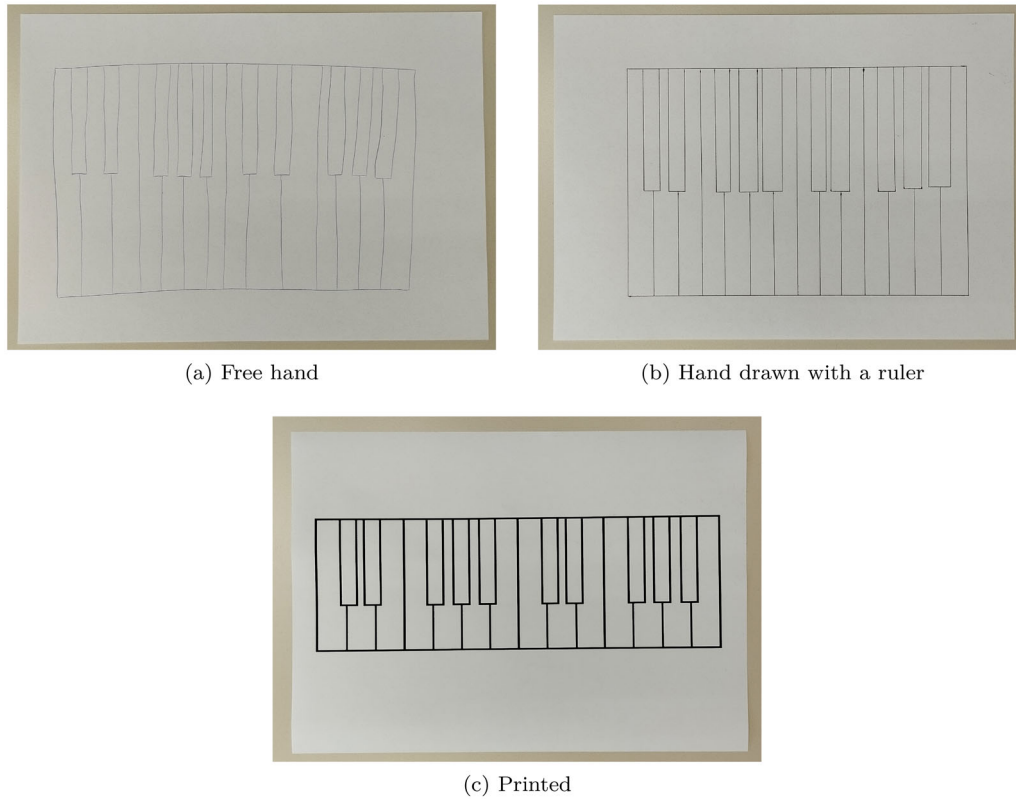


Figure 7. The three kinds of keyboard used for testing.

Table 1. Pixel accuracy of keyboard detection algorithm.

Experiment	Accuracy
Hand-drawn	95.00%
Hand-drawn with ruler	96.74%
Printed	97.69%

The accuracy of the detection algorithm has been calculated with the following formula:

$$\text{accuracy} = \frac{TP}{n} \quad (1)$$

where TP is the number of true positives, defined as the number of pixels correctly classified by the algorithm as belonging to the keyboard, and n is the total number of pixels within the area bounded by the overlay used to mask the image (see Figure 4).

Results are shown in Table 1.

Evidences show the robustness of the Keyboard Detection procedure, with the lowest accuracy being 95% for the hand-drawn keyboard. Furthermore, we empirically observed using the application that missing around 5% or less of the pixels of the keyboard is negligible. Key touches are usually performed around the center of each note, while the missed pixels lie around the boundary of the keys. Thus, users mostly interact with correctly recognized pixels.

4.3. Note detection

This test aims to evaluate whether notes played by a user are correctly recognized by the software. Three kinds of tests have been performed:

- One hand and one finger: each note has been played going from left to right and backward
- Two hands and one finger for each hand: pairs of notes (at a distance of 2 keys from one another) have been played from left to right and backward

- One hand and two fingers: pairs of notes (at a distance of 2 keys from one another) have been played from left to right and backward, adopting the same strategy as the previous test (i.e., the one with two hands and one finger for each hand)

To collect ground truth, recordings of the tests have been manually checked frame by frame. True positives (*TP*), true negatives (*TN*), false positives (*FP*) and false negatives (*FN*) have been calculated and shown in Table 2.

Results of the experiments are shown in Table 3.

The high accuracy of the model indicates that the system mostly recognizes when a user is playing a note or not.

Other metrics scored lower results, around 80%; this can be explained by the camera positioning. Having the camera in front of the hands allows the best view for detecting vertical movement of the fingers, but also could result in the hands being often partially occluded. This leads to MediaPipe struggling to detect the correct hand position.

4.4. UMUX results

To gather information on the perceived usability of the program, 15 people have been invited to participate in the experimentation using the application. Of the 15 people involved, 5 were elementary school kids, from 6 to 9 years old, and the others were adults, from 20 to 35 years old. Among these 15 people, 6 had no musical knowledge, 6 had little musical knowledge, coming mainly from middle school and high school, and 3 were professional piano players. Each one had 5 min to use the application, and had to do both the keyboard detection phase and the playing phase. When time's up, each participant has been asked to fill out the UMUX questionnaire (Usability Metric for User Experience; Finstad, 2010).

Results are shown in Table 4 and compared to those of the previous work (Avola et al., 2023).

The collected findings indicate an overall improvement in the perceived usability of the system from the previous version. The results also show the importance of both improving the algorithms used in the software and having the whole program executed from a smartphone, without additional hardware required.

5. System limitations

MusiKeyrtual makes use of lightweight algorithms for most of its subprocedures, and MediaPipe ML-based hand detection algorithm. Using such simple algorithms, although effective, as the tests have shown, in real-life scenarios can fail. This is a trade-off between having robust algorithms yet

Table 2. Calculation of *TP*, *TN*, *FP*, *FN*.

	User plays	User does not play
Application plays	<i>TP</i>	<i>FP</i>
Application does not play	<i>FN</i>	<i>TN</i>

Table 3. Metrics of the note detection algorithm.

	Percentage (%)
Accuracy	87.40
Precision	80.23
Recall	81.18
F_1	80.70

Table 4. Comparison of the UMUX questionnaire results of both versions of Keyrtual.

	Minimum	Average	Maximum
Keyrtual (previous version)	70.83%	79.86%	87.50%
MusiKeyrtual (presented work)	84.11%	92.28%	97.91%

computationally heavy algorithms, and light ones, which are less accurate, but can run on as many devices as possible.

During the Real-Time Phase the camera is in front of the piano, and depending on how the person plays, part of the hand can get occluded to the camera, making MediaPipe's hand detection inaccurate. This is an inherent limitation of using a single RGB camera, such as the one used on the smartphone.

6. Conclusions

In this work, MusiKeyrtual, a pipeline for playing a piano keyboard drawn on paper, has been proposed. Development has been guided by the objective of having an application that requires only a smartphone to run. For this reason, each component of the pipeline proposed uses algorithms with a low computational cost. The only exception is hand tracking, performed with MediaPipe, which uses a light ML model. MusiKeyrtual has been developed taking into consideration many drawbacks of classical AR systems. Having the whole application running on a single smartphone takes away the need to combine different devices. Also, relying on light algorithms made it so that even cheap smartphones can run the program, and it is not necessary to offload heavy computation to a remote machine, thus requiring no internet access.

All the tests performed indicate positive outcomes. The UMUX questionnaire on perceived usability reflects a high level of user satisfaction, while the other results show the robustness and effectiveness of the algorithms proposed. Future works might take into consideration the improvement of this pipeline, increasing the accuracy of the algorithms, and adopting computationally cheaper models for hand detection. Tests can also be improved by expanding the group of participants. Testing the program with more people would allow us to better understand how certain demographics respond to the application, especially those targeted by this project. Furthermore, other kinds of tests can be carried out in order to acquire qualitative results and gain deeper insights into the usability of the system among the various demographics. Our expectation is that this work might motivate other research groups to develop lightweight algorithms that can be executed on as many low-computationally powerful devices as possible, to extend accessibility of the technology, without the need for expensive and specialized hardware.

Notes

1. Official OpenCV Documentation; OpenCV for Unity Documentation.
2. Samsung's website.
3. Unity's website.

Ethical approval

This research did not involve the collection of sensitive personal data. Participants interacted with the involved experimental phase, and all data collected were anonymized before analysis. As such, formal ethical approval was not required by Sapienza University of Rome.

Informed consent

Participants were informed about the nature of the study prior to their involvement, and their participation was entirely voluntary.

Author contributions

CRedit: **Valerio Venanzi**: Conceptualization, Data curation, Methodology, Software, Validation, Writing – original draft; **Andrea Princic**: Conceptualization, Data curation, Methodology, Software, Validation, Writing – original draft; **Marco Raoul Marini**: Formal analysis, Funding acquisition, Project administration, Supervision, Writing – review & editing; **Gian Luca Foresti**: Supervision, Writing – review & editing; **Luigi Cinque**: Funding acquisition, Supervision, Validation, Writing – review & editing.

Disclosure statement

The authors declare no competing interests. This research did not receive any specific grant from funding agencies in the public, commercial, or non-profit sectors. The authors have no affiliations with or involvement in any organization or entity with a financial interest, or conflict of interest, with the subject matter or materials discussed in this manuscript.

Funding

The research leading to these results has received funding from the project “DISEGNO + SMARTPHONE = MUSICA” financed by Fondazione TIM in the Call for Ideas 2023 context.

ORCID

Valerio Venanzi  <http://orcid.org/0009-0006-2847-4998>

Andrea Princic  <http://orcid.org/0009-0004-5063-1115>

Marco Raoul Marini  <http://orcid.org/0000-0002-2540-2570>

Gian Luca Foresti  <http://orcid.org/0000-0002-8425-6892>

Luigi Cinque  <http://orcid.org/0000-0001-9149-2175>

References

- Archana, R., & Jeevaraj, P. S. E. (2024). Deep learning models for digital image processing: A review. *Artificial Intelligence Review*, 57(1), 11. <https://doi.org/10.1007/s10462-023-10631-z>
- Arena, F., Collotta, M., Pau, G., & Termine, F. (2022). An overview of augmented reality. *Computers*, 11(2), 28. <https://doi.org/10.3390/computers11020028>
- Avola, D., Cinque, L., Foresti, G. L., & Marini, M. R. (2019). An interactive and low-cost full body rehabilitation framework based on 3D immersive serious games. *Journal of Biomedical Informatics*, 89, 81–100. <https://doi.org/10.1016/j.jbi.2018.11.012>
- Avola, D., Cinque, L., Marini, M. R., Princic, A., Venanzi, V. (2023, September 25–28). *Keyrtual: A lightweight virtual musical keyboard based on RGB-D and sensors fusion* [Paper presentation]. Computer analysis of images and patterns: 20th International Conference, CAIP 2023, Limassol, Cyprus, Proceedings, Part II (pp. 182–191). Springer-Verlag. https://doi.org/10.1007/978-3-031-44240-7_18
- Canny, J. (1986). A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6), 679–698. <https://doi.org/10.1109/TPAMI.1986.4767851>
- Caselli, N. K., Sehyr, Z. S., Cohen-Goldberg, A. M., & Emmorey, K. (2017). ASL-LEX: A lexical database of American sign language. *Behavior Research Methods*, 49(2), 784–801. <https://doi.org/10.3758/s13428-016-0742-0>
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). *Imagenet: A large-scale hierarchical image database* [Paper presentation]. 2009 IEEE Conference on Computer Vision and Pattern Recognition, Miami, FL (pp. 248–255).
- Deng, Y. (2019). Deep learning on mobile devices: A review. In S. S. Agaian, V. K. Asari, & S. P. DelMarco (Eds.), *Mobile multimedia/image processing, security, and applications 2019* (Vol. 10993, pp. 109930A). SPIE. <https://doi.org/10.1117/12.2518469>
- De Smedt, Q., Wannous, H., & Vandeborre, J.-P. (2016). *Skeleton-based dynamic hand gesture recognition* [Paper presentation]. 2016 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), Las Vegas, NV (pp. 1206–1214).
- Fillwalk, J. (2015). *Chromachord: A virtual musical instrument* [Paper presentation]. 2015 IEEE Symposium on 3D User Interfaces (3DUI), Arles (pp. 201–202).
- Finstad, K. (2010). The usability metric for user experience. *Interacting with Computers*, 22(5), 323–327. <https://doi.org/10.1016/j.intcom.2010.04.004>
- Fischler, M. A., & Bolles, R. C. (1981). Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6), 381–395. <https://doi.org/10.1145/358669.358692>
- Fontana, F., Lanzino, R., Diko, A., Foresti, G. L., Cinque, L. (2024). *Cyclebnn: Cyclic precision training in binary neural networks*. <https://arxiv.org/abs/2410.00050>
- Fontana, F., Lanzino, R., Marini, M. R., Avola, D., Cinque, L., Scarcello, F., & Foresti, G. L. (2024). Distilled gradual pruning with pruned fine-tuning. *IEEE Transactions on Artificial Intelligence*, 5(8), 4269–4279. <https://doi.org/10.1109/TAI.2024.3366497>

- Foxlin, E., & Harrington, M. (2000). *Weartrack: A self-referenced head and hand tracker for wearable computers and portable VR* [Paper presentation]. Digest of Papers. Fourth International Symposium on Wearable Computers, Atlanta, GA (pp. 155–162).
- Gil-Martín, M., Marini, M. R., San-Segundo, R., & Cinque, L. (2024). Dual leap motion controller 2: A robust dataset for multi-view hand pose recognition. *Scientific Data*, 11(1), 1102. <https://doi.org/10.1038/s41597-024-03968-9>
- Guo, L., Lu, Z., & Yao, L. (2021). Human-machine interaction sensing technology based on hand gesture recognition: A review. *IEEE Transactions on Human-Machine Systems*, 51(4), 300–309. <https://doi.org/10.1109/THMS.2021.3086003>
- Jasim, M., Zhang, T., & Hasanuzzaman, M. (2014). A real-time computer vision-based static and dynamic hand gesture recognition system. *International Journal of Image and Graphics*, 14(01n02), 1450006. <https://doi.org/10.1142/S0219467814500065>
- Jiang, D., Li, G., Sun, Y., Kong, J., & Tao, B. (2019). Gesture recognition based on skeletonization algorithm and cnn with asl database. *Multimedia Tools and Applications*, 78(21), 29953–29970. <https://doi.org/10.1007/s11042-018-6748-0>
- Kumar, R., Kumbharkar, P., Vanam, S., & Sharma, S. (2023). Medical images classification using deep learning: A survey. *Multimedia Tools and Applications*, 83(7), 19683–19728. <https://doi.org/10.1007/s11042-023-15576-7>
- Lanzino, R., Fontana, F., Diko, A., Marini, M. R., & Cinque, L. (2024). *Faster than lies: Real-time deepfake detection using binary neural networks* [Paper presentation]. IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024 - Workshops, Seattle, WA, USA, June 17–18, (pp. 3771–3780). IEEE. <https://doi.org/10.1109/CVPRW63382.2024.00381>
- Liang, H., Wang, J., Sun, Q., Liu, Y.-J., Yuan, J., Luo, J., & He, Y. (2016). Barehanded music: Real-time hand interaction for virtual piano. In *Proceedings of the 20th ACM SIGGRAPH symposium on interactive 3D graphics and games* (pp. 87–94). Association for Computing Machinery. <https://doi.org/10.1145/2856400.2856411>
- Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Grundmann, M. (2019). Mediapipe: A framework for building perception pipelines. CoRR, abs/1906.08172. <http://arxiv.org/abs/1906.08172>
- Marini, M. R., Mecca, A., Foresti, G. L., Cinque, L. (2024). A natural interaction system for medical training through VR technology. In G. Ochoa-Ruiz, et al. (Eds.), *37th IEEE International Symposium on Computer-Based Medical Systems, CBMS 2024, Guadalajara, Mexico, June 26–28, 2024* (pp. 237–242). IEEE. <https://doi.org/10.1109/CBMS61543.2024.00047>
- Masoumian, A., Rashwan, H. A., Cristiano, J., Asif, M. S., & Puig, D. (2022). Monocular depth estimation using deep learning: A review. *Sensors*, 22(14), 5353. <https://doi.org/10.3390/s22145353>
- Mohamed, N., Mustafa, M. B., & Jomhari, N. (2021). A review of the hand gesture recognition system: Current progress and future directions. *IEEE Access*, 9, 157422–157436. <https://doi.org/10.1109/ACCESS.2021.3129650>
- Prashanth, A., Jayalakshmi, S. L., & Vedhapriyavadhana, R. (2024). A review of deep learning techniques in audio event recognition (AER) applications. *Multimedia Tools and Applications*, 83(3), 8129–8143. <https://doi.org/10.1007/s11042-023-15891-z>
- Qi, J., Ma, L., Cui, Z., & Yu, Y. (2024). Computer vision-based hand gesture recognition for human-robot interaction: A review. *Complex & Intelligent Systems*, 10(1), 1581–1606. <https://doi.org/10.1007/s40747-023-01173-6>
- Sarao, V., Veritti, D., De Nardin, A., Misciagna, M., Foresti, G., & Lanzetta, P. (2023). Explainable artificial intelligence model for the detection of geographic atrophy using colour retinal photographs. *BMJ Open Ophthalmology*, 8(1), e001411. <https://doi.org/10.1136/bmjophth-2023-001411>
- Suzuki, S., & Be, K. (1985). Topological structural analysis of digitized binary images by border following. *Computer Vision, Graphics, and Image Processing*, 30(1), 32–46. [https://doi.org/10.1016/0734-189X\(85\)90016-7](https://doi.org/10.1016/0734-189X(85)90016-7)
- Wohlgenannt, I., Simons, A., & Stieglitz, S. (2020). Virtual reality. *Business & Information Systems Engineering*, 62(5), 455–461. <https://doi.org/10.1007/s12599-020-00658-9>
- Younesi, A., Ansari, M., Fazli, M., Ejlali, A., Shafique, M., & Henkel, J. (2024). A comprehensive survey of convolutions in deep learning: Applications, challenges, and future trends. *IEEE Access*, 12, 41180–41218. <https://doi.org/10.1109/ACCESS.2024.3376441>
- Zhang, L., Zhu, G., Mei, L., Shen, P., Shah, S. A. A., & Bennamoun, M. (2018). Attention in convolutional LSTM for gesture recognition. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, & R. Garnett (Eds.), *Advances in neural information processing systems* (Vol. 31). Curran Associates, Inc. <https://proceedings.neurips.cc/paperfiles/paper/2018/file/287e03db1d99e0ec2edb90d079e142f3-Paper.pdf>
- Zhou, J., & Hoang, J. (2005). *Real time robust human detection and tracking system* [Paper presentation]. 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)-Workshops (pp. 149–149).
- Zottin, S., De Nardin, A., Colombi, E., Piciarelli, C., Pavan, F., & Foresti, G. L. (2024). U-DIADS-BIB: A full and few-shot pixel-precise dataset for document layout analysis of ancient manuscripts. *Neural Computing and Applications*, 36(20), 11777–11789. <https://doi.org/10.1007/s00521-023-09356-5>

About the authors

Valerio Venanzi earned BSc/MSc (cum laude) in Computer Science from Sapienza University Rome (2021/23). Since 2024, he is a PhD student focused on graph theory algorithms for efficient computation on low-end devices and real-world applications.

Andrea Princic received BSc and MSc (cum laude) in Computer Science from Sapienza University of Rome, Italy, in 2021 and 2023 respectively. His interests lie in teaching, AR development and Human-Computer Interaction. His current work focuses on efficient algorithms for low-end devices and applications for educational purposes.

Marco Raoul Marini earned BSc/MSc (cum laude) Computer Science from Sapienza University Rome (2015), then a PhD (2019). He researches Human Behavior Analysis and Machine Learning within Extended Reality (XR), focusing on VR/BCI interactions, locomotion, and human-centered systems. Member of IEEE/ACM/CVPL.

Gian Luca Foresti directs a department at University of Udine, leading conferences (IEEE ICIP05, ICIAP11, AVSS11). His research focuses on computer vision/AI and multisensor fusion. He authored 400+ papers and won Best Paper Awards (IEEE Vehicular Electronics, ICDSC 2010/2019, and others). IAPR fellow, Senior IEEE member, CVPL president.

Luigi Cinque earned a Physics degree (1983), developed expert systems (Alenia S.p.A., 1984-1990). Now Full Professor at Sapienza University Rome, researching distributed video analysis and multisensor fusion for surveillance, autonomous vehicles, human behavior understanding. Author of 200+ papers; reviewer for IEEE journals & EU programs. Senior Member IEEE, ACM, IAPR.